

Esperanto SoC Session - I

Developer: Ryan Naveed
Dated: July 30th, 2026

- [illegible]

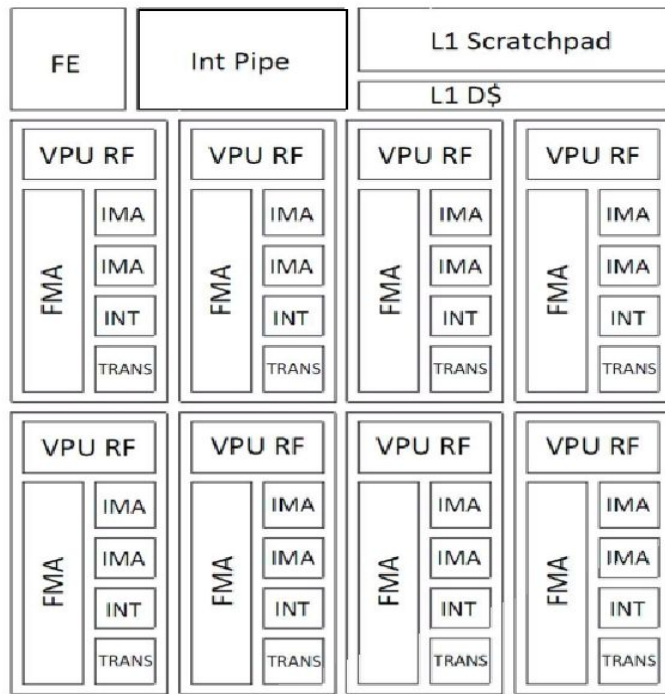
Core SoC and IO Overview

1. 1,093 total RV64 RISC-V cores
 - a. 1,088 In-Order cores (Minions) for ML computes
(1024 cores for processing, 32 cores for task division, and the remaining 32 cores for consistent silicon yield)
 - b. 4 Out-of-Order cores (Maxions) for management tasks
 - c. 1 dedicated RISC-V service processor
2. IOShire subsystem integrating UART, SPI, I2c, I3C, USB, and other peripheral controllers
3. PCIe Shire handling an 8-lane Gen4 physical layer with two dual-mode controllers.



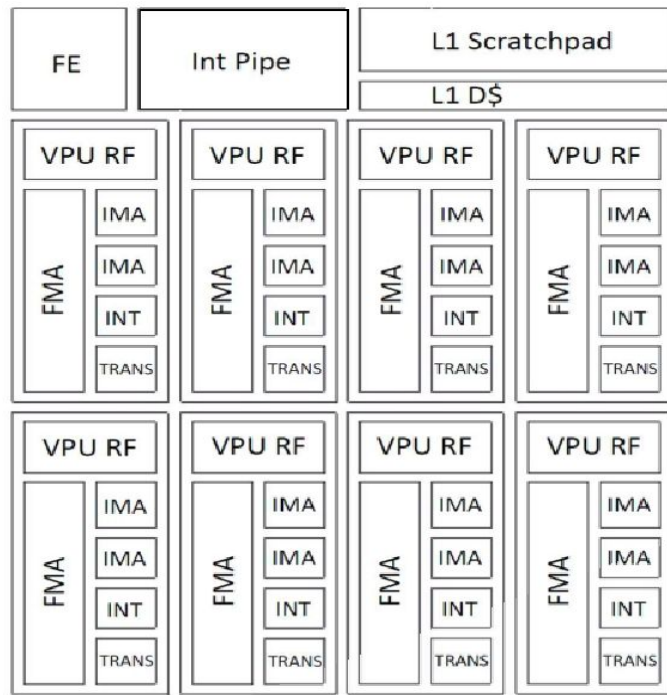
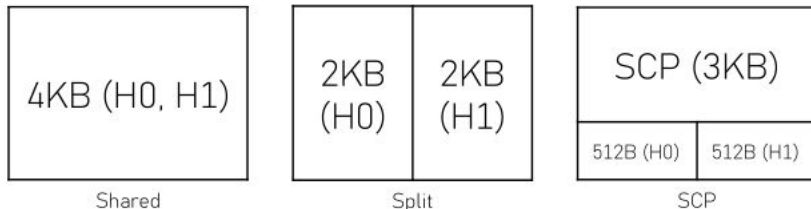
ET-Minion Core Architecture

- 64-bit RISC-V In-Order Processor
- Supports RV64IMFC ISA Extensions
- Dual Hardware Threads per Core
- Single-Issue Integer Execution Pipeline
- Private 4 KB L1 Data Cache
- Integrated 8-Lane SIMD Vector Processing Unit (VPU)
- Custom Esperanto SIMD extension



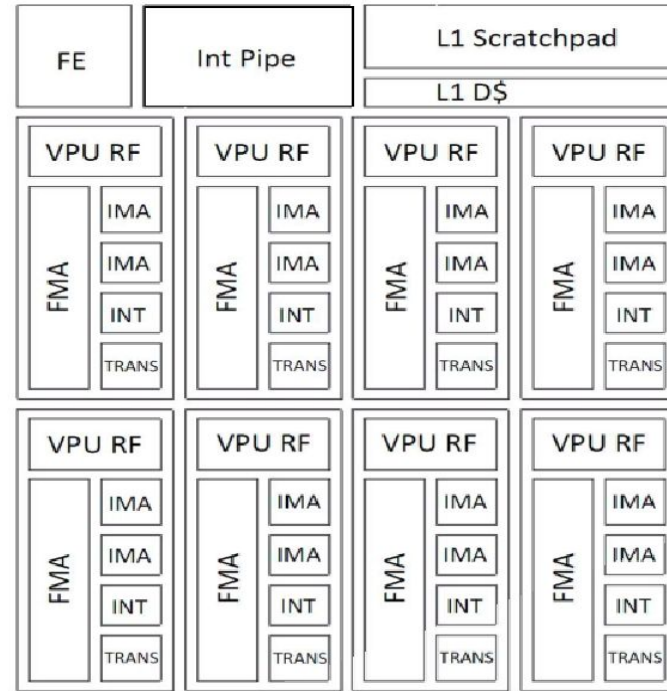
L1 - Data Cache Configuration

- 4KB set-associative cache size
- 16 sets, 4 ways, 64-bytes cache lines
- Non-coherent
- Configuration through **mcache_control** CSR
- Configurable modes:



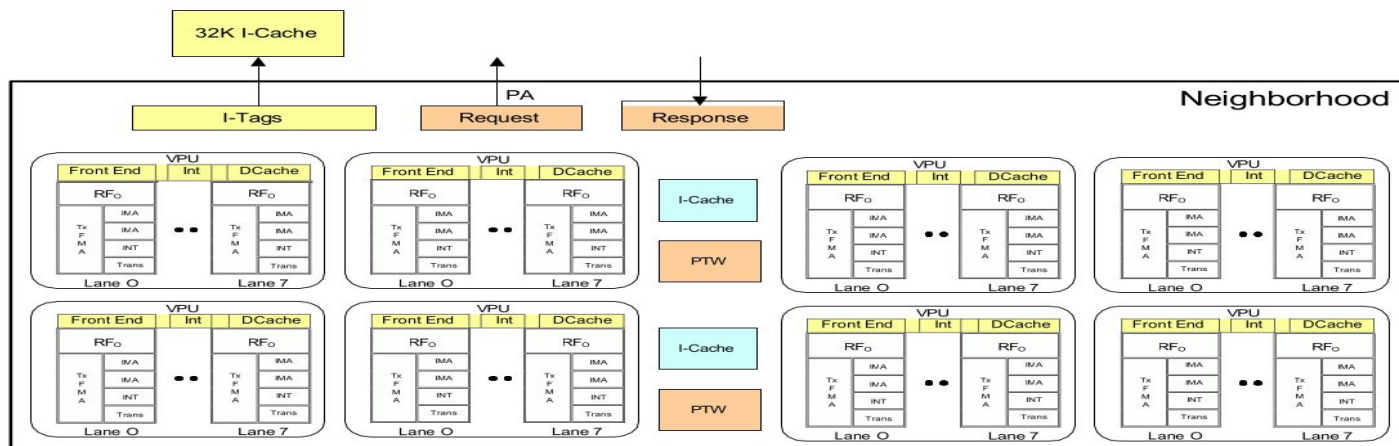
Vector Processing Unit (VPU)

- 8 identical Synchronous Lanes
- 32 x 256-bit Vector Registers (per thread)
- **TxFMA Unit** – Executes one 32-bit or two 16-bit floating-point fused multiply-add operations
- **Dual IMA Units** – Performs four 8-bit integer multiple-accumulate (MAC) operations
- **INT Unit** – Executes Integer arithmetic and logical operations
- **TRANS Unit** – Supports Exp2, Log2 and Reciprocal operations

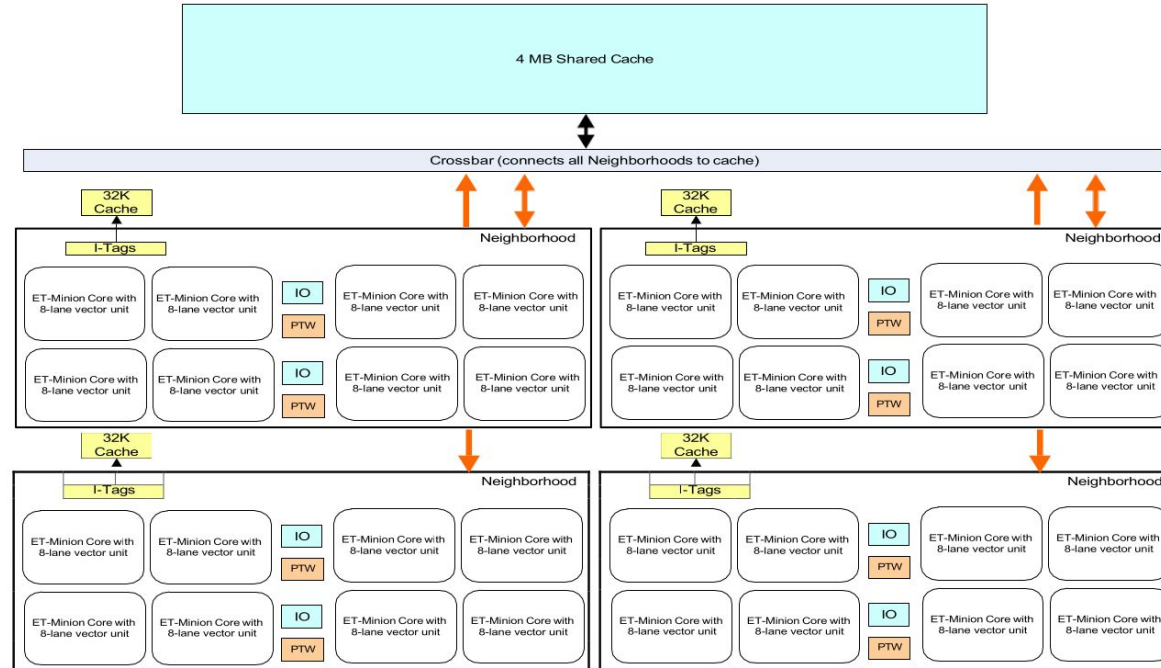


Minion Neighborhood Overview

- Cluster of 8-Minion cores
- 32 KB shared L1 instruction cache
- Dual shared L0 microcaches and PTW

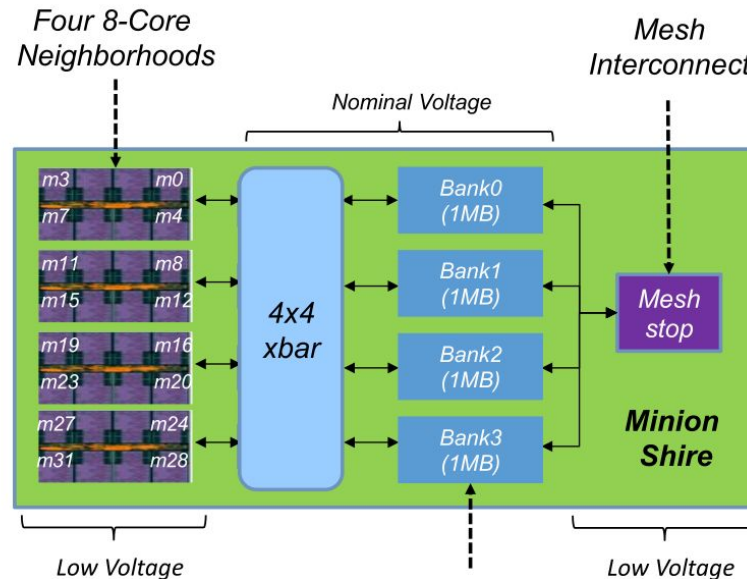


Minion Shire Hierarchy



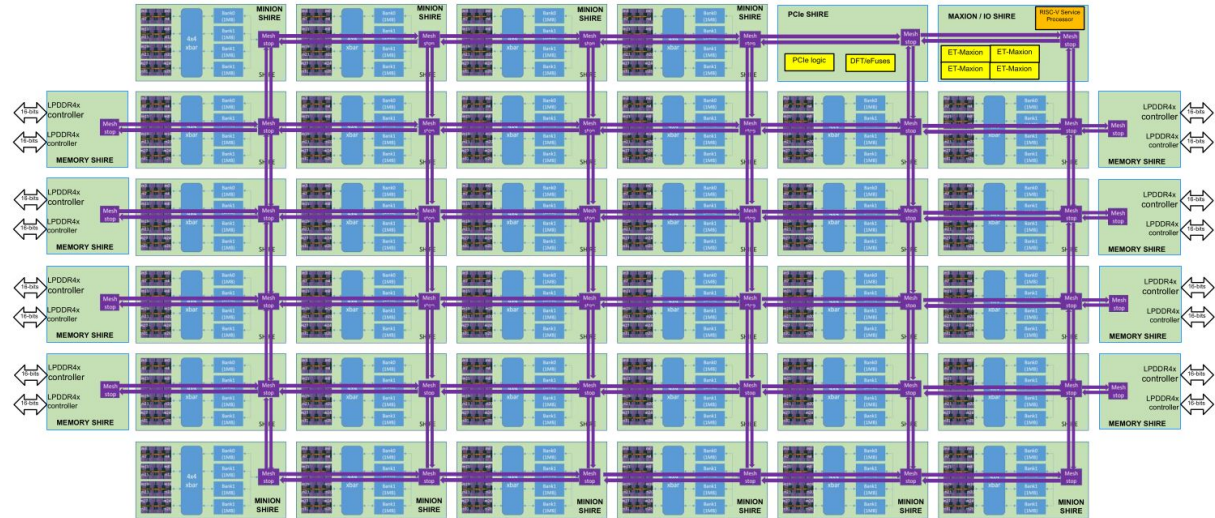
Minion Shire Overview

- Cluster of 32-Minion cores
- Four interconnected Neighborhoods
- Integrated 4MB shared Shire Cache (L2/L3)
- Integrated 4×4 crossbar for Neighborhood communication

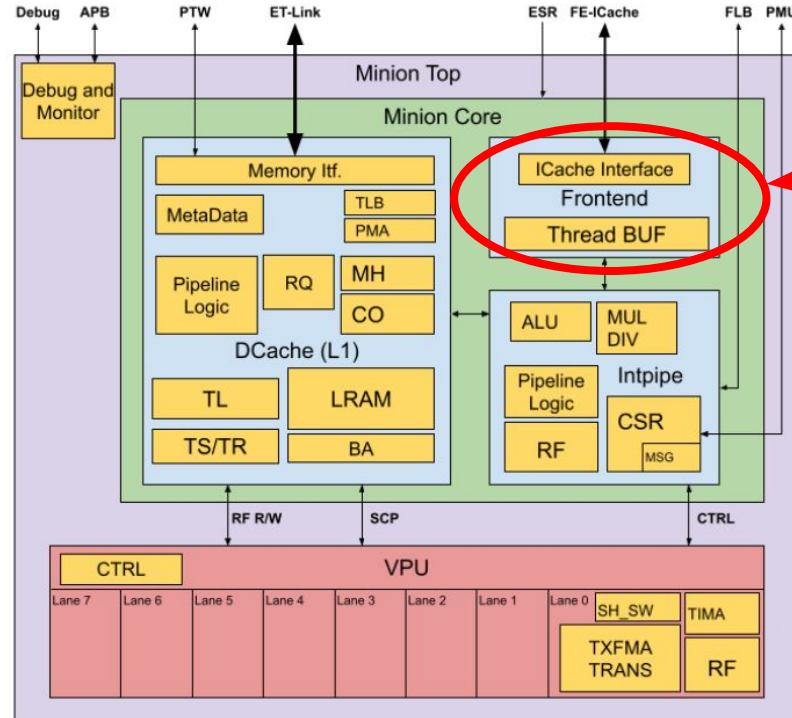


Mesh Network-on-Chip (NoC)

- Top-level 8x6 grid interconnect
- 44-Mesh stops
- Corner mesh stops are unoccupied



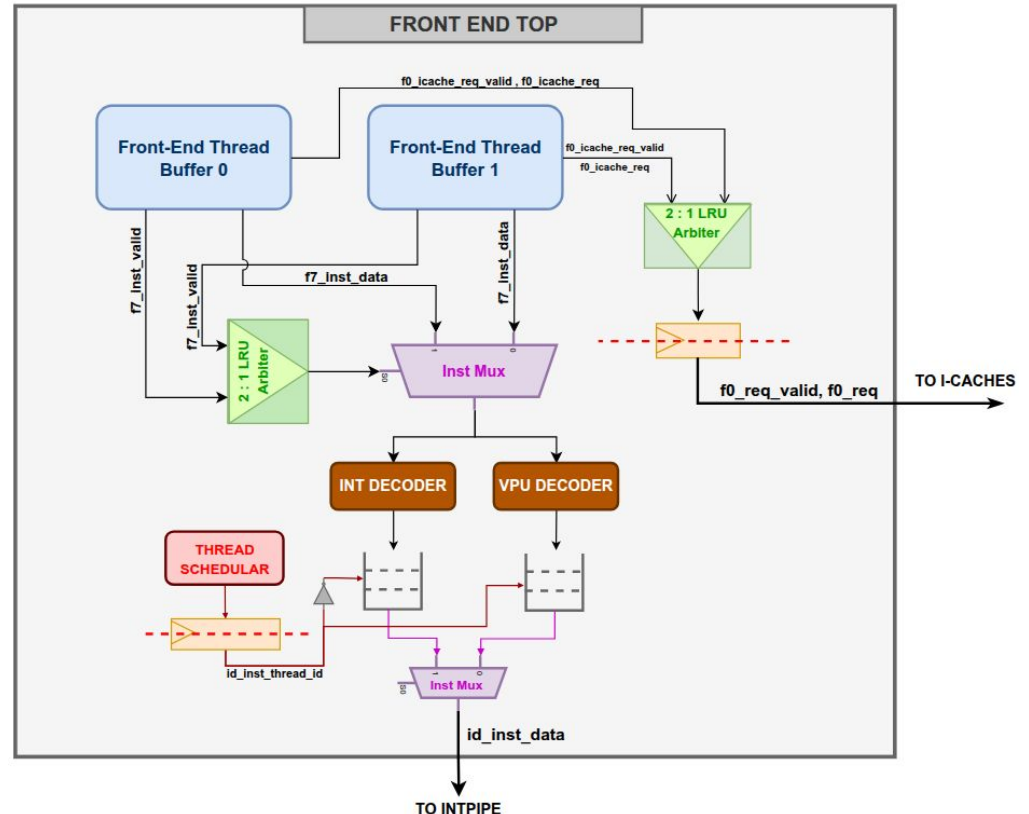
Minion Architectural Diagram



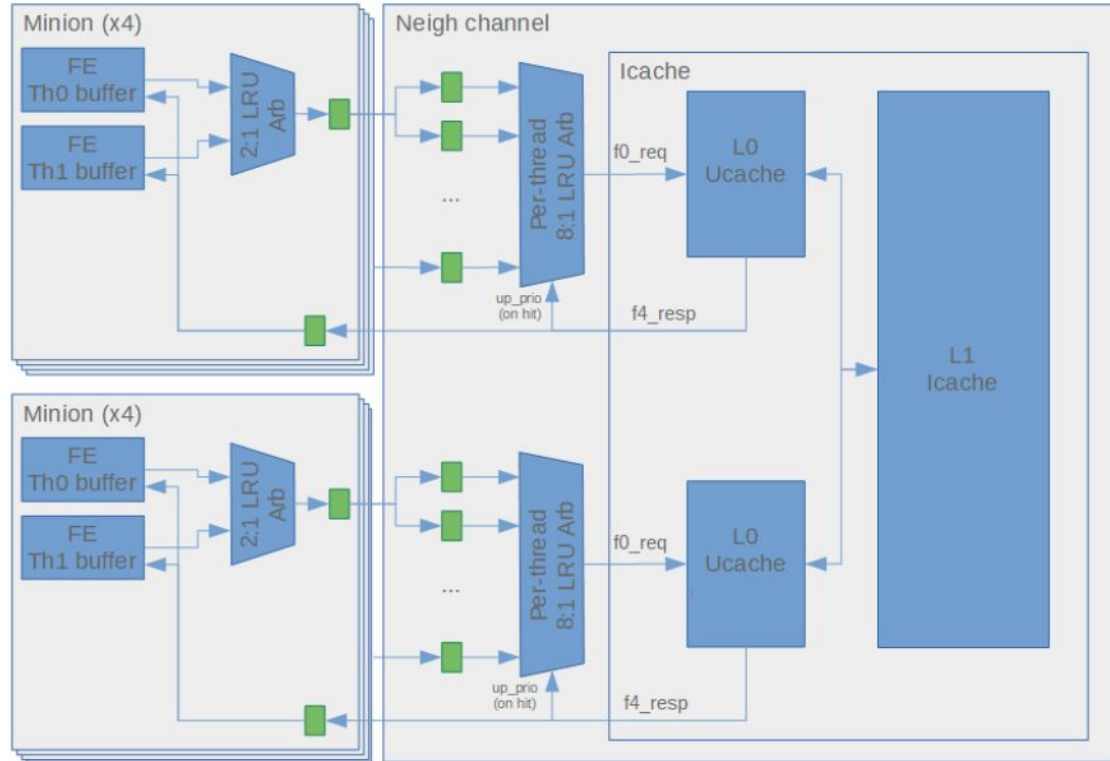
Front-End Unit for fetching and decoding instructions

FrontEnd Unit Internal Blocks

1. **Front-End Buffer x 2:** Buffer to cache instructions from L0 u-cache.
2. **Program Counter x 2:** Separate PC for each thread (instantiated in FE Buffer).
3. **Thread Scheduler:** Schedules the thread to select the instruction to be passed to the next module (INTPIPE).
4. **IntPipe Decoder:** decodes the read instruction for the INTPIPE module.
5. **VPU Decoder:** decodes the read instruction for the VPU module.
6. **FIFO x 2:** FIFO is used to store both INTPIPE and VPU decoded signals, useful when an instruction gets fetched from the buffer during the pipeline stall.



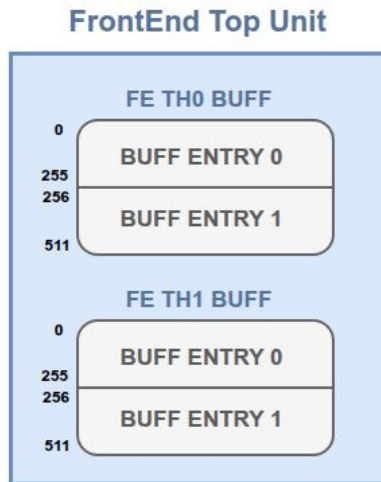
FrontEnd I-Cache Communication Interface Diagram



Thread FrontEnd Buffer

- Each thread has its own **FE buffer** containing two entries each of width 256 bits.
- Each entry is filled with instructions at consecutive addresses.
- The read data width from the I-cache is 32 bytes (256 bits or half a cache line).
- Dedicated Program Counter (PC) in each FE Buffer, tracks the next instruction to be executed for that thread.
- The whole buffer gets invalidated upon receiving a new PC address from the INTPIPE, due to branch/jump instruction. In this case no instructions would get fetched until we received a valid response from L0 u-cache.

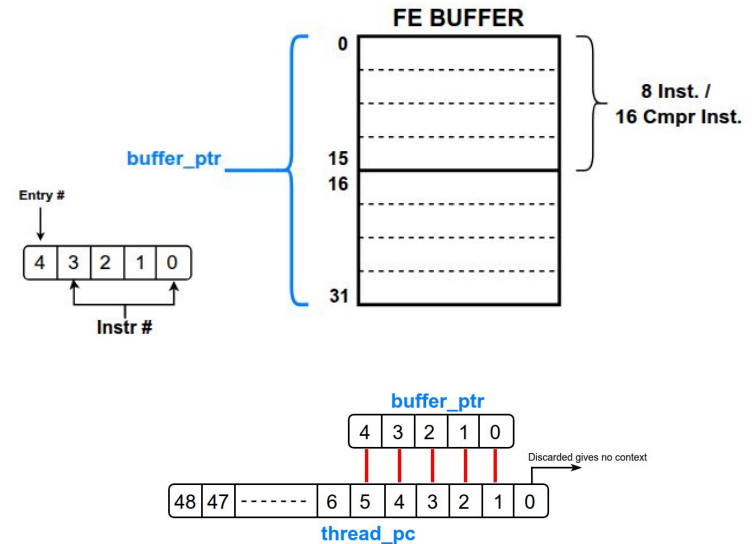
Optimizations to be made: Rather than invalidating the buffer directly, we should first determine whether the buffer contains the instructions pointing to the new address.



FrontEnd Buffer Instruction Fetch

Each buffer entry can hold eight general instructions or sixteen compressed instructions.

- **buffer_ptr** reads instructions from the FE Buffer. The MSB selects the buffer entry, and the lower four bits point to a compressed instruction.
- **buffer_ptr** value is derived from the Thread PC located in this FrontEnd Buffer. The bits (5:1) are copied in this pointer.
- The fetched instruction is of 32 bits.
- The instruction retrieved from the address is always of size:
 $\text{buffer_ptr} = (\text{Instruction}[1:0] == 2'b11)?$
 $\text{buffer_ptr} + 2 : \text{buffer_ptr} + 1$

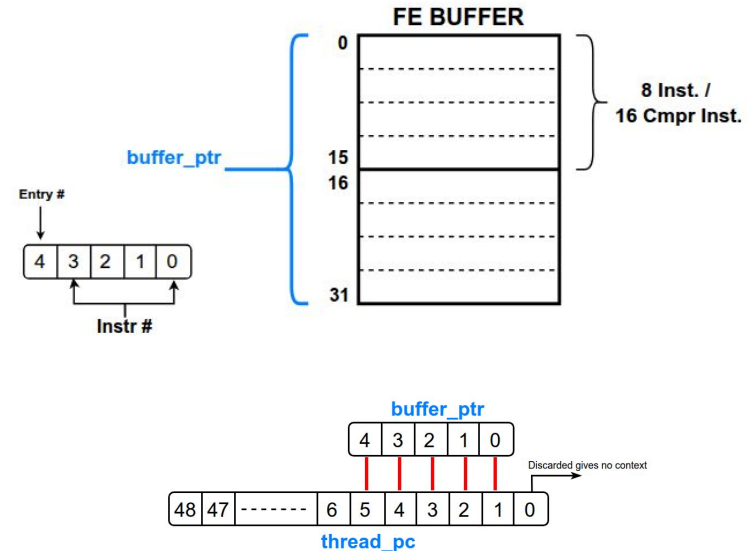


FrontEnd Buffer Entry Invalidation

The entry got cleared/invalidated in 4 cases, and a new fetch request gets generated for that entry.

- INTPIPE sends a new PC address in case of branch/jump or interrupts/exceptions.
- PC ends with 5'h1C (decimal 28), and the instruction is not compressed.
- PC ends with 5'h1E (decimal 30), and the instruction is compressed.
- PC ends with 5'h1E (decimal 30), and the instruction is not compressed, and the next entry is valid.

When executing instructions from one entry, if the other is empty or invalid, generate a request in parallel to retrieve the next consecutive instructions in that entry.



Buffer Entry Fill Example

[m , n] represents the address of the next 8 instructions in the corresponding buffer entry. The following mechanism is used to fill the buffer entries.

1. Master Shire enables the minion and provides a starting address "A".
Address A $\Rightarrow$ [x , x]
1. Request sent to I-Cache
Address {A & ~0x1F} $\Rightarrow$ [A>>5 , x]
1. It creates a new request since there is a free entry.
Address {(A & ~0x1F) + 32} $\Rightarrow$ [A>>5 , (A>>5)+1]
4. The entry is invalidated (for non-RVC Instruction) upon reading the final instruction from the first entry PC ({(A & ~0x1F) + 28}).
The first entry is removed $\Rightarrow$ [x , (A>>5)+1]
5. It creates a new request to I-Cache since there is once more a free entry.
Address {(A & ~0x1F) + 64} $\Rightarrow$ [(A>>5)+2 , (A>>5)+1]

Buffer Entry Fill Example

[m , n] represents the address of the next 8 instructions in the corresponding buffer entry. The following mechanism is used to fill the buffer entries.

1. Master Shire enables the minion and provides a starting address
Address A $\Rightarrow$ [x , x]
1. Request sent to I-Cache
Address {A & ~0x1F} $\Rightarrow$ [A>>5 , x]
1. It creates a new request since there is a free entry.
Address {(A & ~0x1F) + 32} $\Rightarrow$ [A>>5 , (A>>5)+1]
4. The entry is invalidated (for non-RVC Instruction) upon reading the final instruction from the first entry PC ({(A & ~0x1F) + 28}).
The first entry is removed $\Rightarrow$ [x , (A>>5)+1]
5. It creates a new request to I-Cache since there is once more a free entry.
Address {(A & ~0x1F) + 64} $\Rightarrow$ [(A>>5)+2 , (A>>5)+1]



Live-Lock: A single instruction infinite loop where the instruction is the last one in the cache line and the jump is repeatedly mispredicted.

FrontEnd Stages (8 Stages, F0 .. F7)



F0 Stage:

Handler for INTPIPE request and Thread enable/disable. I-cache requests are generated based on the INTPIPE_PC or THREAD_PC.

F1–F5 Stage:

Wait 5 cycles to receive a response from L0 u-cache.

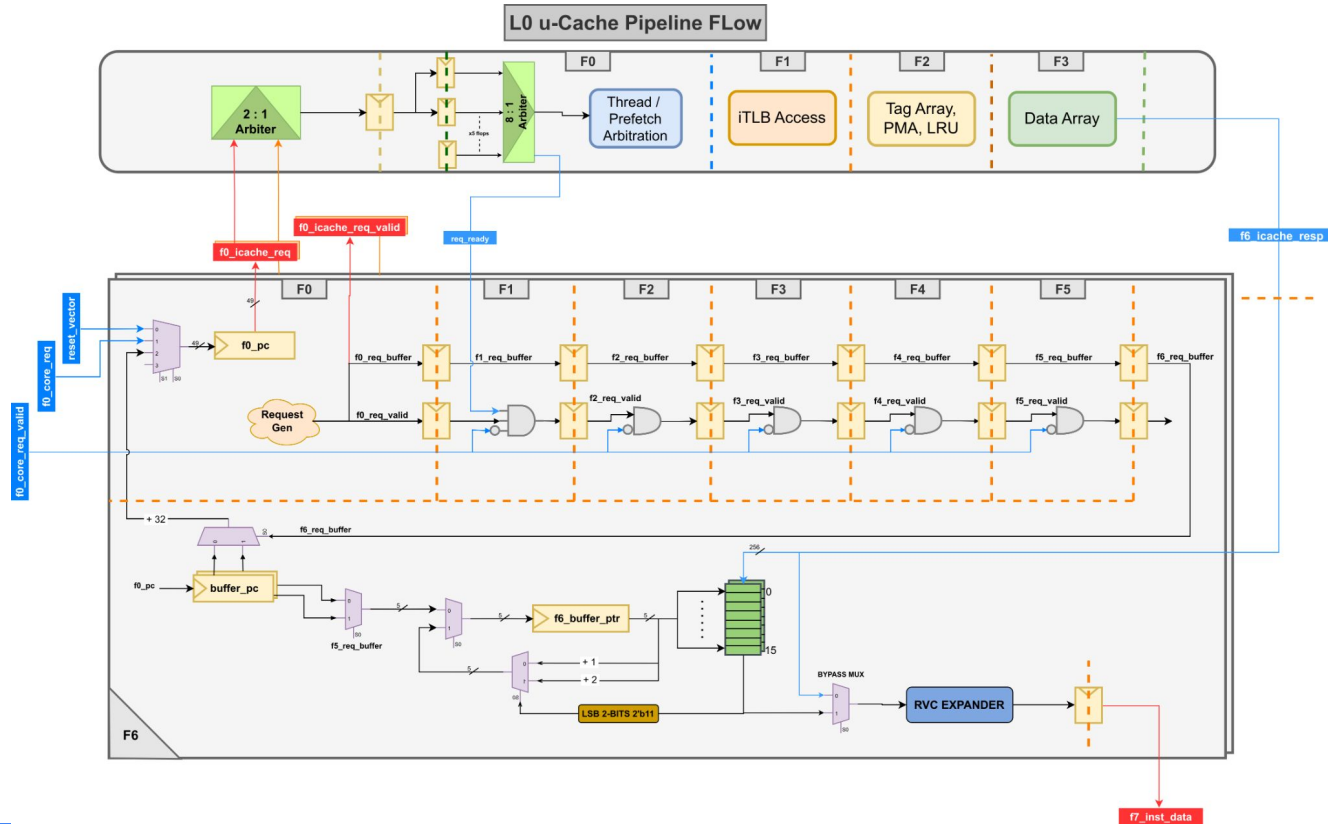
F6 Stage:

- Stores the I-cache response in FrontEnd buffer.
- Read instructions from the FrontEnd buffer.

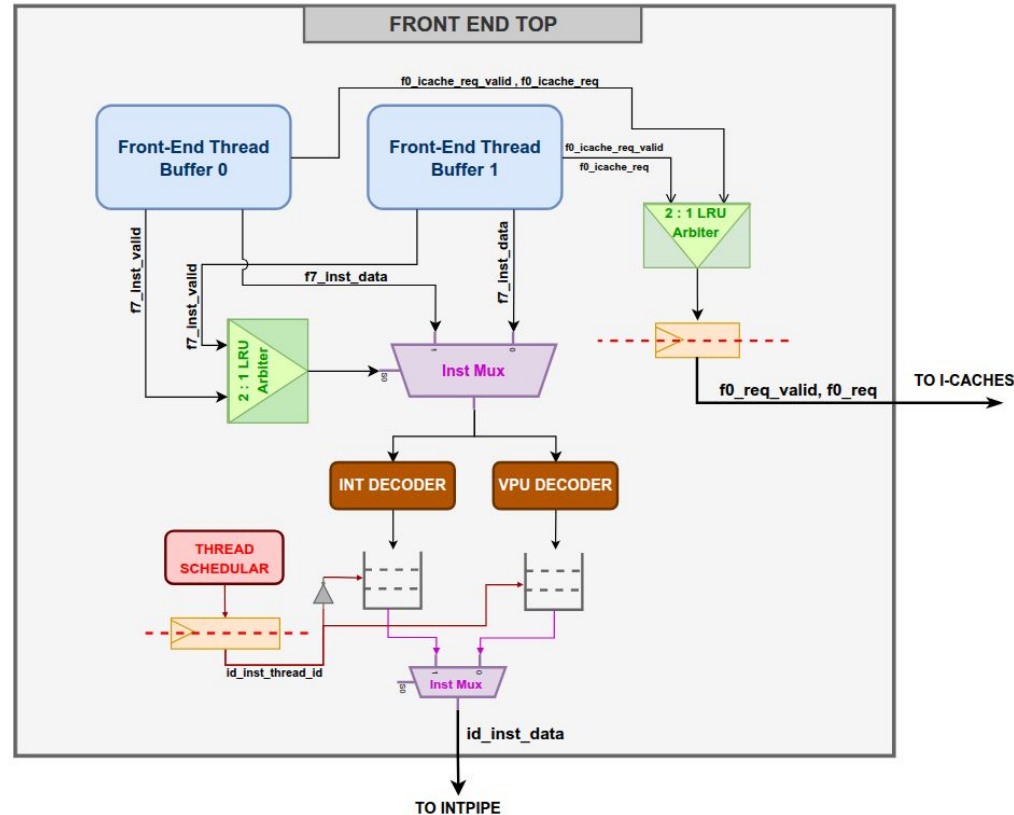
F7 Stage:

- Scheduling of Threads
- INTPIPE and VPU decoder
- Read/Write instruction control signals FIFOs.

FrontEnd Stages (8 Stages, F0 .. F7)



FrontEnd Last Stage (Instruction Decode)



FrontEnd Thread Scheduler RTL

The scheduling is done using the Round-Robin algorithm.

If only one thread is valid, select it; if both are valid, pass the thread that wasn't selected in the previous cycle.

A thread's state is marked VALID when an instruction is read from its PC, there's no new request from INTPIPE, and its FIFO has space.

```
module frontend_thread_sched (
    // System signals
    input logic clock,
    input logic reset,

    // Thread enable and valid signals
    input logic [CORE_NR_THREADS-1:0] f7_valid_tid,
    input logic [CORE_NR_THREADS-1:0] f0_thread_enabled,
    input logic [CORE_NR_THREADS-1:0] id_core_stall,

    // Picked thread
    output logic [CORE_NR_THREADS-1:0] f7_thread_away,
    output logic f7_thread_id
);

////////////////////////////////////////
// Select thread to propagate
// When both threads are valid, access them alternatively
// Otherwise, read the one which is valid
////////////////////////////////////////
logic f7_thread_id_prev; // previously selected thread
logic [CORE_NR_THREADS-1:0] f7_valid_tid_qual; // Inst valid in F4 for each thread
logic [CORE_NR_THREADS-1:0] f7_thread_away_next;

// CLK RST EN DOUT DIN DEF
`RST_EN_FF(clock, reset, |f7_valid_tid_qual, f7_thread_id_prev, f7_thread_id, 1'b0)
`RST_FF (clock, reset, f7_thread_away, f7_thread_away_next, 2'b11)

always_comb begin
    // Computes the qualified valid per thread
    f7_thread_away_next = f0_thread_enabled & ~id_core_stall; // Thread can fetch
    f7_valid_tid_qual = f7_valid_tid & f7_thread_away; // Valid from fetch, thread awake

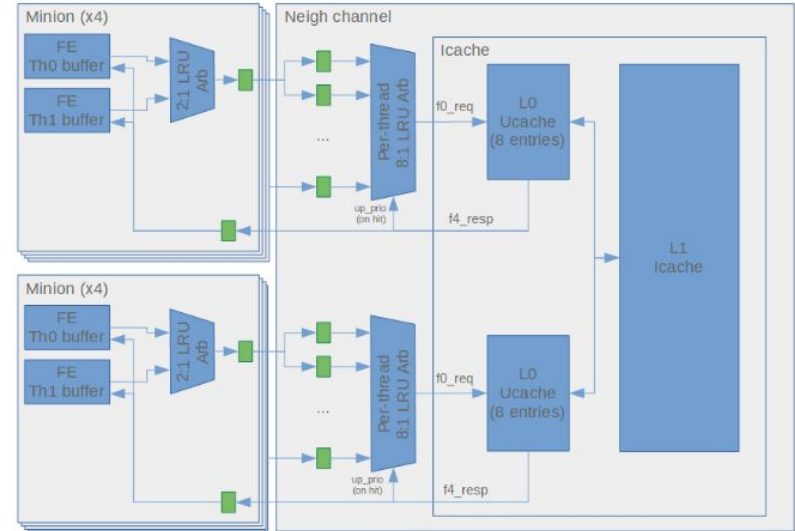
    // Thread ID picker

    // Selects thread ID for current cycle based on valids and picker
    if (!f7_valid_tid_qual[1]) f7_thread_id = 1'b0; // If no inst for thread 1, select 0
    else if (!f7_valid_tid_qual[0]) f7_thread_id = 1'b1; // If no inst for thread 0, select 1
    else f7_thread_id = ~f7_thread_id_prev; // Alternate threads otherwise
end // always_comb

endmodule
```

FrontEnd I-Cache Communication Interface

1. One of the FrontEnd thread buffer requests reaches the L0 u-cache via the LRU arbiter.
2. After 5 cycles, the FrontEnd receives the response in both cases of HIT and MISS.
3. In the HIT case, the response contains instructions to be stored in the corresponding FrontEnd buffer, and the transaction is complete.
4. For MISS responses, the corresponding requesting thread is put to sleep, and L0 sends a fetch request to the L1 cache. Once the L1 cache responds, a "fill done" signal is sent to all minions, allowing them to wake up and retry again.
5. The LRU arbiter only updates the priority on the cache HIT, allowing the miss thread to retry with a higher priority when it is woken up.



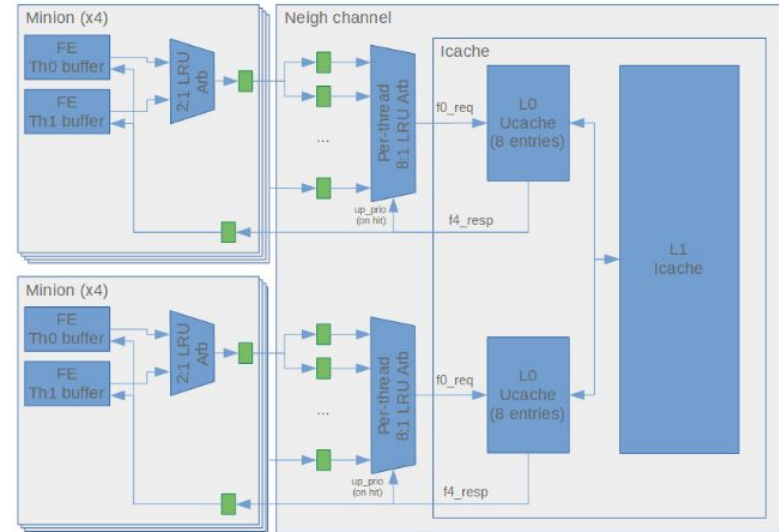
FrontEnd Issues 1 - Unnecessary Buffer Flush

- We take a backward branch for this matrix multiplication code after each Multiply the operation of accumulation.
- Although the prior instruction might still be accessible and might not need to be flushed for each Branch/Jump instruction, this would flush out both Buffer entries containing the instructions.

```
// conducts AxB matrix multiplication, stores result into C
void matrix_mult(int A[M][N], int B[N][P], int C[M][P])
{
    for (int i = 0; i < M; i++)
    {
        for (int j = 0; j < P; j++)
        {
            int accum = 0;
            for (int k = 0; k < N; k++)
            {
                accum += A[i][k] * B[k][j];
            }
            C[i][j] = accum;
        }
    }
}
```

FrontEnd Issue 2 - Unnecessary utilization of req line

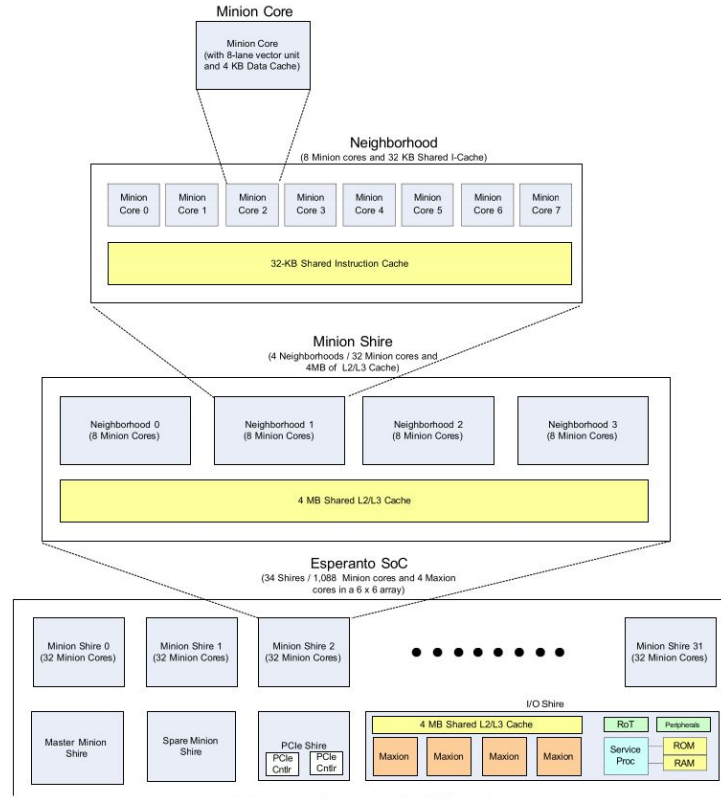
- In this case, when the 2:1 arbiter grants a THREAD request for the request line, it occupies the flop until it is accepted by the next 8:1 arbiter.
- This introduces unnecessary latency because we split the request to their respective flops, and the previous flop should be allocated to the other THREAD for requesting a new cache request.



FrontEnd Issue 3 - Waiting for the request response in case of request KILL

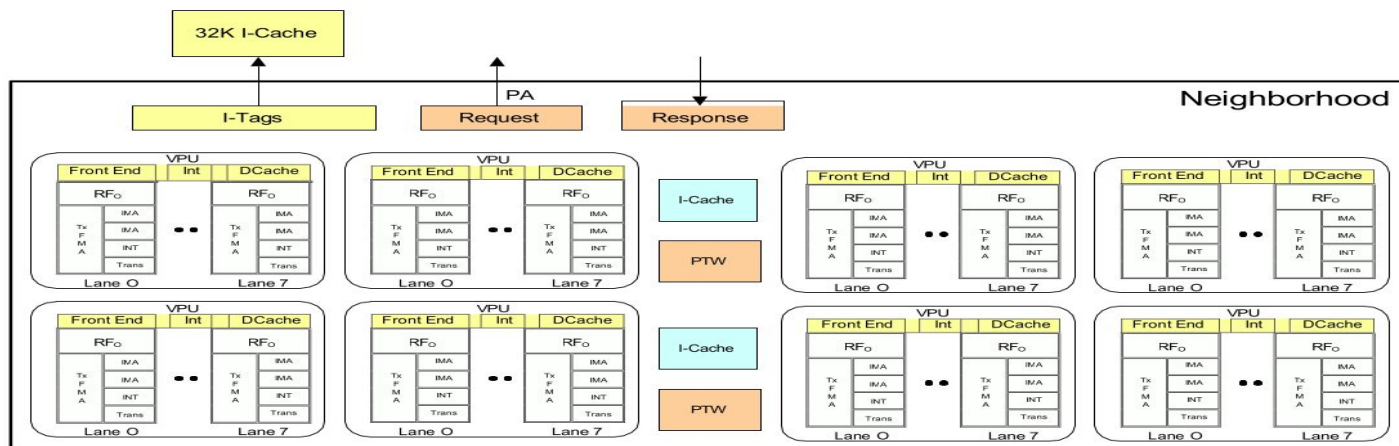
- The Front-End Thread Buffer kills the current request when
 - INTPIPE provides a new PC address due to branch/jump.
 - Current request haven't yet received by the L0 u-cache.
- But, Instead of passing an additional Invalidate signal up to the arbiter flops, it first waits for the prior request to get accepted, ignores the granted response, and then issue a new request with the updated PC address.

Esperanto SoC Hierarchy



Minion Neighborhood Overview

- Cluster of 8-Minion cores
- 32 KB shared L1 instruction cache
- Dual shared L0 microcaches and PTW



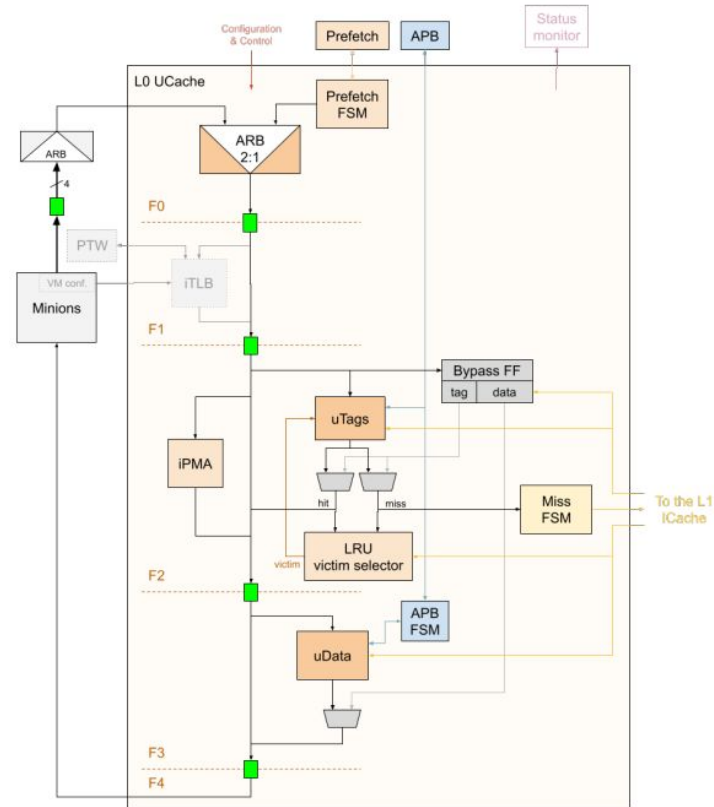
L0 u-cache Overview



- **Capacity & Structure:** 16-entry, Fully Associative
- **Addressing:** VIPT (Virtually Indexed, Physically Tagged)
- **Pipeline:** 5-stage non-blocking (F0–F4) latency
- **Instances:** 2x L0 Micro-caches per Neighborhood
- **Service:** Shared by 4 Minions (8 Threads)
- **Bypass Capability:** Supports a bypass path that allows instruction fetches to be serviced directly from the L1 I-Cache without allocating the data into the L0 u-cache.
- **Software-Controlled Prefetching:** Provides a software-controlled prefetch mechanism capable of preloading up to 64 consecutive cache lines into the L1 I-Cache.

L0 u-cache Internal Blocks

1. **iTLB:** 16-entry virtual-to-physical address translation cache (2 entries/thread).
2. **μTags:** Stores tags for all 16 L0 Micro-cache entries.
3. **iPMA:** Validates physical address access attributes.
4. **LRU Victim Selector:** Selects replacement way and updates LRU on hits, misses, and L1 fills.
5. **μData:** Stores 16 cache lines, 64 bytes per line



L0 u-cache Five Pipeline Stages

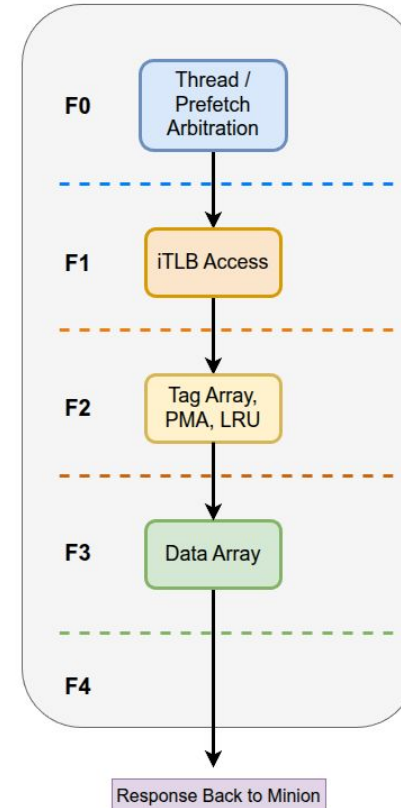
F0 Stage: Arbitration and Request selection

F1 Stage: Address Translation (TLB Lookup)

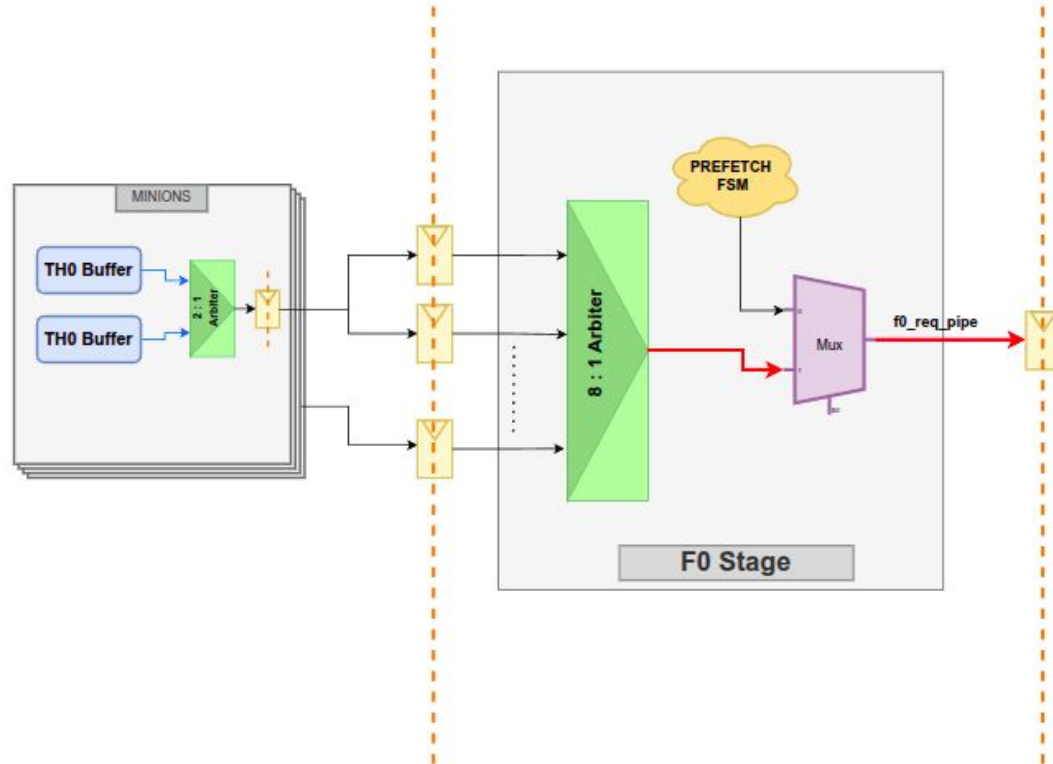
F2 Stage: Tag Lookup, PMA check and LRU Update.

F3 Stage: Data Array access.

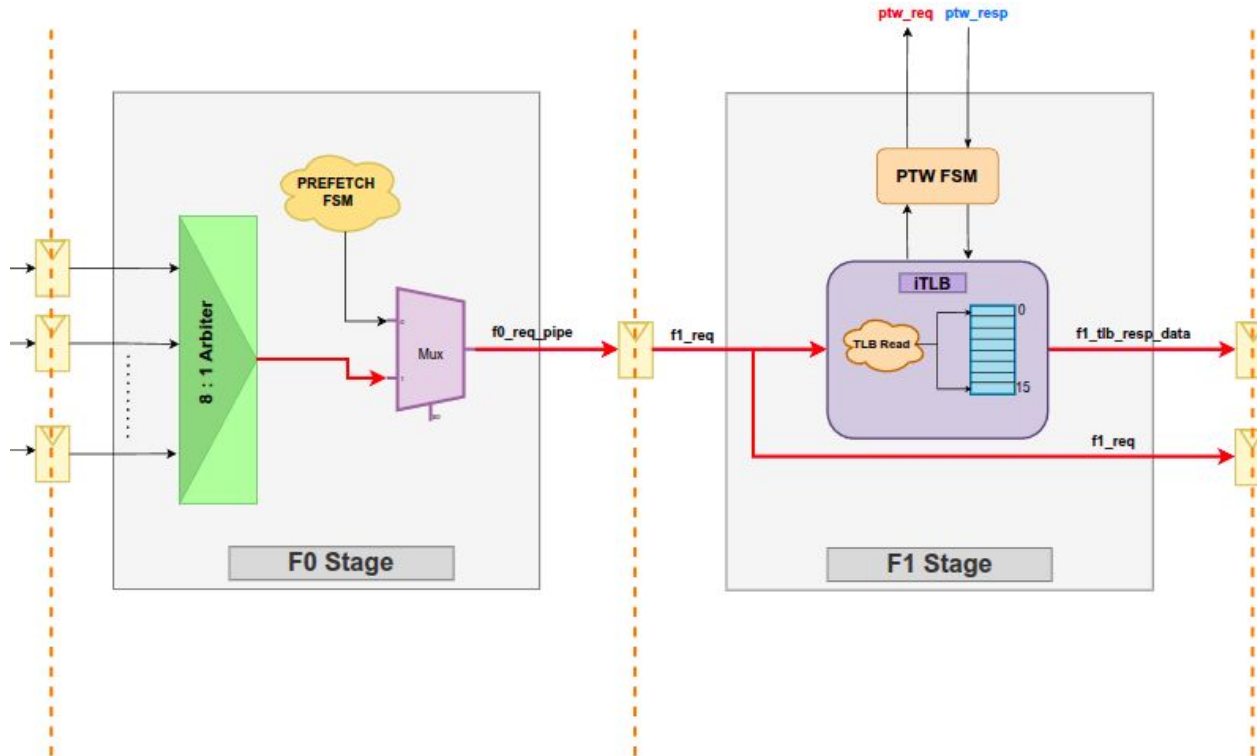
F4 Stage: Response back to Minion.



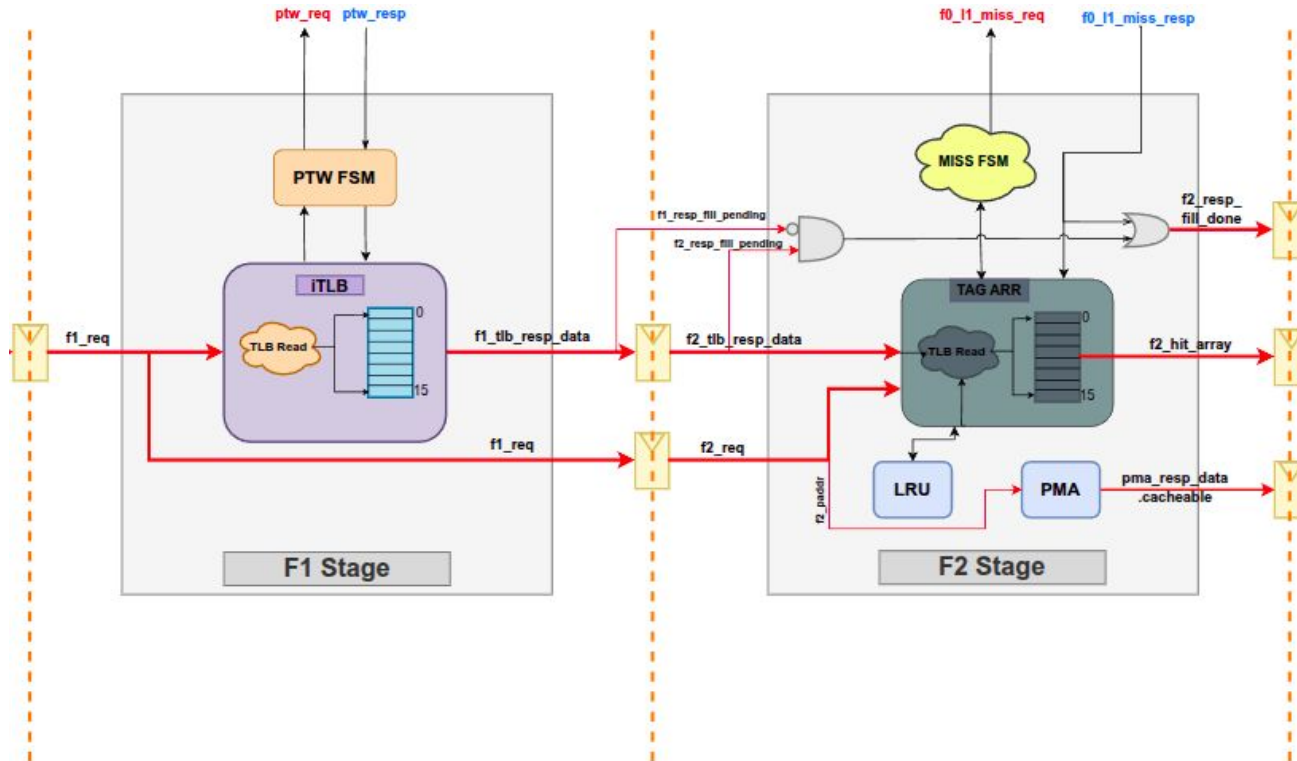
L0 u-Cache HIT Flow: Cycle 0 Arbitration



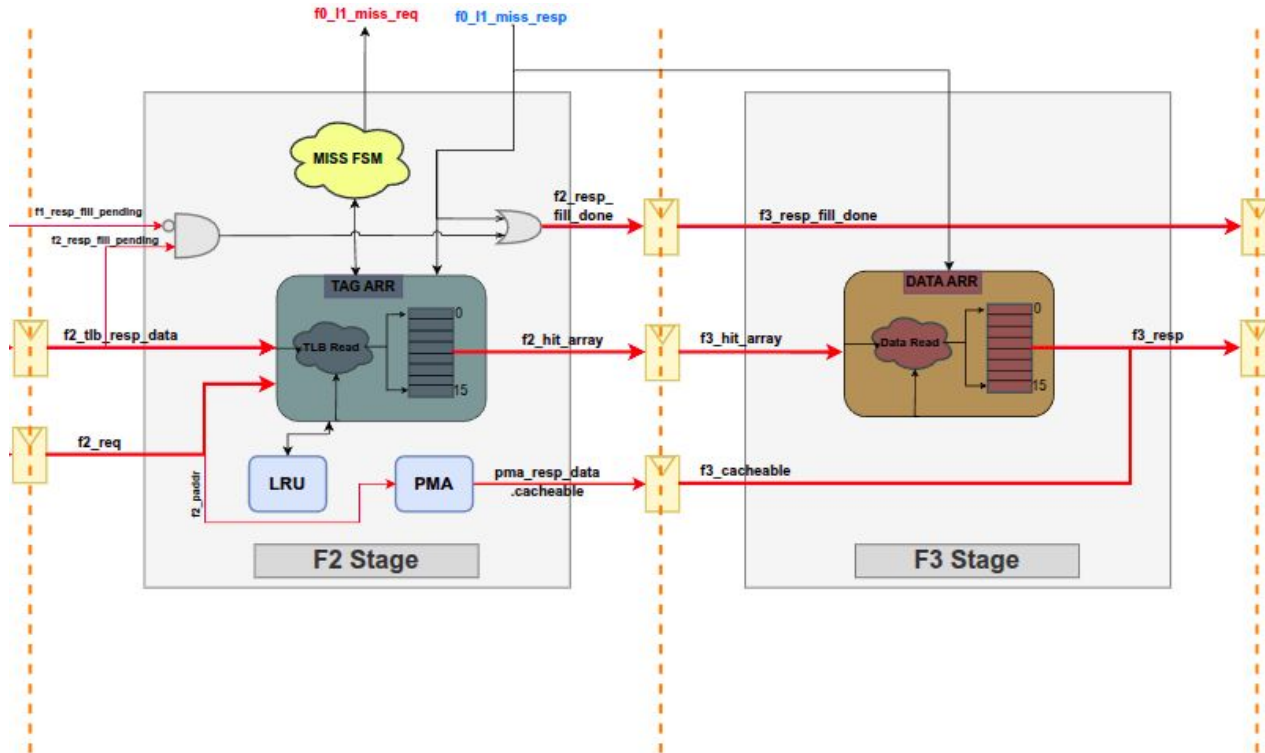
L0 u-Cache HIT Flow: Cycle 1 TLB Lookup Up



L0 u-Cache HIT Flow: Cycle 2 TAG, PMA, LRU



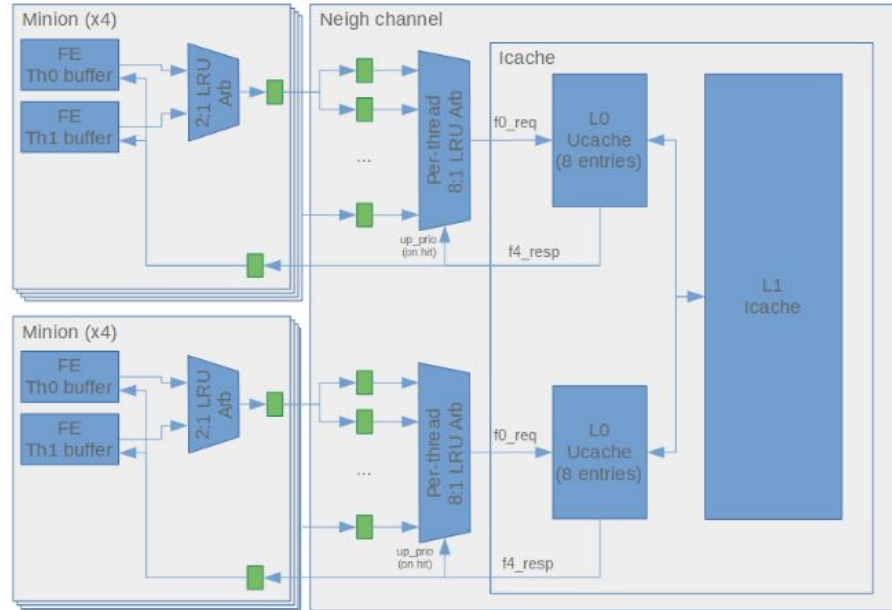
L0 u-Cache HIT Flow: Cycle 3 Data Access



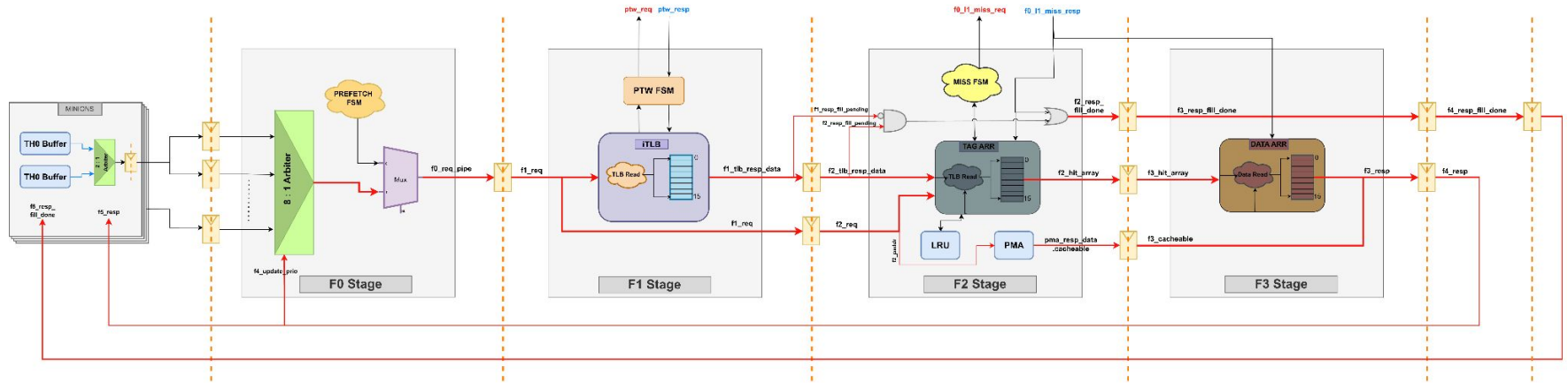
L0 u-Cache HIT Flow: Cycle 4 Update Arbiter LRU

If the “f4_resp” is HIT,
then the arbiter updates
the LRU policy of that
thread.

No updation would be
made for cache miss
response



L0 u-Cache Response Complete Flow:



TLB Miss Handling

Cycle	Operation
F0	Arbitration selects the thread request.
F1	TLB lookup. On a miss, a PTW request is generated immediately and tlb_fill_pending is asserted
F2 – F3	If tlb_fill_pending remains asserted, the miss response continues through the pipeline.
F3 – F4	The requesting thread receives a MISS response and transitions to the sleep state.

- PTW request is issued immediately upon TLB miss.
- The lookup pipeline is not stalled.
- The miss response follows the normal 5-stage latency.
- The requesting thread sleeps after receiving the miss response.

TLB Fill Completion & Thread Wake-up

PTW Response Flow

1. PTW returns the translated address.
2. TLB entry is updated.
3. **tlb_fill_pending** is cleared.
4. **resp_fill_done** is generated.
5. **resp_fill_done** propagates to the 6th stage.
6. Sleeping threads wake up and retry their requests.
7. **resp_fill_done** is intentionally delayed until the 6th stage to prevent overlap between the sleep and wake-up events.

TAG Array Miss Flow

Cycle	Operation
F0	Arbitration
F1	TLB lookup (Hit)
F2	Tag Array lookup (Miss) and L1 I-Cache request generated
F3 – F4	Miss response propagates through the pipeline. Requesting thread receives MISS response and enters sleep.

L1 I-Cache response & Thread Wake-up

I-Cache Response Flow

1. L1 I-Cache returns the requested cache line.
2. Tag Array and Data Array are updated.
3. **resp_fill_done** is generated and propagated to the 6th stage.
4. Sleeping threads wake up and retry and again go to sleep if the response fill is not relevant to that thread.

Note:

- **resp_fill_done** and **I0_cache_response** are common for all 8 threads.
- The normal response (resp) is also observed by all threads, with each thread acting only on the response intended for it.

Live-Lock Timing Diagram

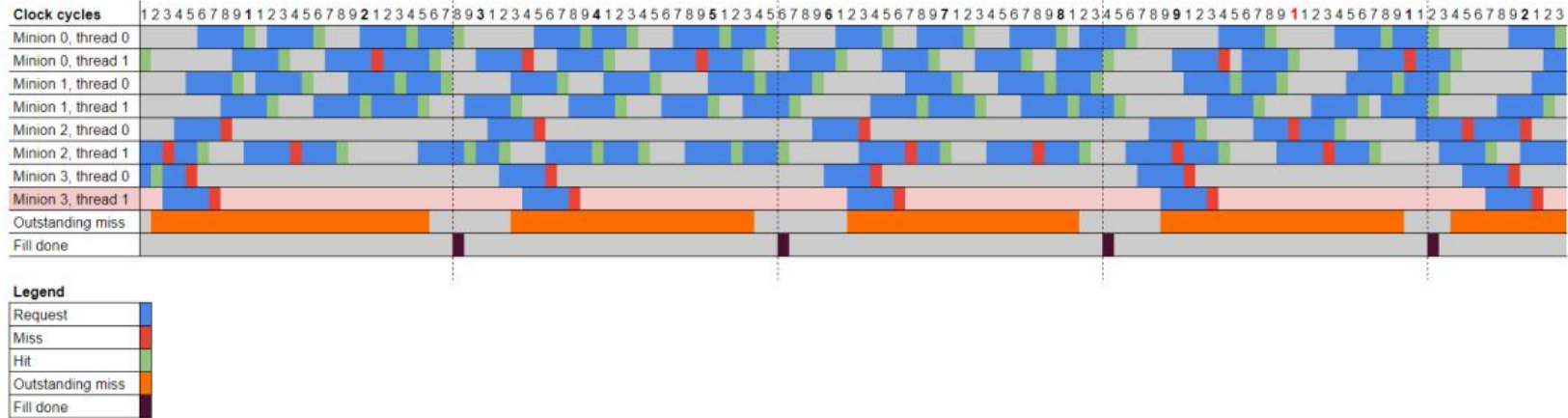


Figure 2 - Timing Diagram of an ICache Livelock Scenario

Software Pre-Fetch Mechanism

- The **NORTH** L0 micro-cache includes a software-controlled prefetch engine.
- Prefetch requests target the L1 I-Cache.
- A single prefetch operation can fetch up to 64 consecutive cache lines.
- Prefetching begins when the **esr_prefetch_start** signal is asserted.
- If a new **esr_prefetch_start** request arrives while a previous prefetch sequence is still active, then the current prefetch operation is aborted, and the FSM immediately starts the new prefetch sequence.
- During the prefetch all requests would intentionally get missed, and the L1 I-cache response would not be cached.

Software Pre-Fetch FSM

State	Operation
IDLE	Waiting for <code>esr_prefetch_start</code> .
REQUEST	Generates the next prefetch request when no thread request is pending.
CHECK_HIT	TLB LookUp to obtain the Physical address
FILL_WAIT	Waiting for the L1 I-Cache to return <code>resp_valid</code>
FILLDONE_WAIT	Waiting for <code>resp_fill_done</code> before issuing the next prefetch.
PREREQ_PAUSE	Waits for 8 cycles before issuing the next prefetch request.

Software Pre-Fetch FSM

